

Análise de Código Java

Algoritmo MergeSort — Explicação Técnica

Tempo	Espaço	Paradigma	Linguagem
$O(n \log n)$	$O(n)$	Divisão e Conquista	Java

1. Objetivo Geral

O código implementa o algoritmo **MergeSort** em Java, cujo objetivo é ordenar um array de inteiros em ordem crescente. No método `main`, o array `{5, 2, 8, 1, 9, 3}` é passado ao algoritmo e, ao final, impresso já ordenado como `[1, 2, 3, 5, 8, 9]`.

2. Lógica e Algoritmo

O algoritmo segue o paradigma de **Divisão e Conquista** em três etapas: **(1) Dividir** — o método `mergeSort` calcula o ponto médio `mid = (l + r) / 2` e chama a si mesmo recursivamente para as metades esquerda e direita, até que cada subarray tenha um único elemento (caso base: `l >= r`). **(2) Conquistar** — cada chamada retorna subvetores já ordenados. **(3) Combinar** — o método `merge` copia o trecho do array para um vetor temporário e intercala os dois lados comparando elemento a elemento, reescrevendo o array original em ordem.

Trecho central do método `merge`:

```
int[] tmp = Arrays.copyOfRange(arr, l, r + 1);
int i = 0, j = mid - l + 1, k = l;
while (i <= mid-l && j <= r-l)
    arr[k++] = tmp[i] <= tmp[j] ? tmp[i++] : tmp[j++];
```

3. Conceitos Java Envolvidos

- **Recursão:** `mergeSort` chama a si mesmo para dividir o problema em subproblemas menores.
- **Arrays utilitários:** `Arrays.copyOfRange()` e `Arrays.toString()` da biblioteca padrão `java.util`.
- **Passagem por referência:** O array é modificado in-place (exceto pelo buffer temporário).
- **Operador ternário:** Usado de forma compacta na comparação de elementos durante a intercalação.

4. Complexidade

A **complexidade de tempo** é **$O(n \log n)$** em todos os casos (melhor, médio e pior), pois o array é sempre dividido ao meio e cada nível da recursão processa `n` elementos no total. A

complexidade de espaço é $O(n)$, devido ao vetor temporário alocado em cada chamada de merge — somando todos os níveis, o consumo máximo simultâneo é proporcional a n .

5. Decisões de Projeto e Limitações

- **Decisão:** uso de índices (l , r) em vez de copiar subarrays inteiros nas chamadas recursivas — evita alocações desnecessárias na fase de divisão.
- **Limitação:** o algoritmo opera apenas sobre `int[]`; não usa Generics, portanto não é reutilizável para outros tipos sem refatoração.
- **Limitação:** aloca um novo array temporário em cada chamada de merge, gerando pressão no GC para arrays muito grandes; uma abordagem alternativa seria usar um único buffer auxiliar global.
- **Estabilidade:** o uso de `<=` na comparação garante que o algoritmo seja **estável** (elementos iguais preservam a ordem relativa original).